PROJECT PLAN & SPEC

복지제도 판정 AI 챗봇 (WELFARE AI)

> RAG 기반 사회복지 챗봇 — 프로젝트 기획·계획서
> : 처음 개발하는 사람을 위한 흐름 가이드

문서 목적 — 기획 배경, 개발 계획, 시스템 구성, 필요 프로그램·기능을 한 문서로 정리해 개발 착수 흐름을 잡는다

대상 — 개인 포트폴리오 프로젝트 (특정 채용공고·마감일에 종속되지 않음)

기반 문서 — RAG_사회복지_챗봇개발가이드.md (상세 기술 가이드는 해당 문서 참고)

핵심 원칙 — 검색이 아니라 판정. 환경 세팅보다 리스크 스파이크가 먼저.

1 프로젝트 기획서

1.1 문제 정의

복지제도는 종류가 많고 자격 기준이 복잡해서, 실제로 받을 자격이 있는 사람도 자신이 대상인지 몰라서 신청하지 못하는 경우가 흔합니다. 정보가 없는 게 아니라 **정리되지 않은 방대한 문서(사업안내 지침서) 속에 흩어져 있는 것**이 문제입니다.

1.2 무엇을 만드는가

사용자가 자연어로 자기 상황(나이, 가구원수, 소득, 재산 등)을 설명하면, 대화를 통해 조건을 모으고 **실제로 자격을 계산해서 판정**한 뒤, 관련 근거 문서와 신청 절차까지 안내하는 챗봇입니다.

> **예시 대화**
> 사용자: "70세인데 혼자 살고 있고 월 소득이 80만원 정도예요. 받을 수 있는 지원이 뭐가 있을까요?"
> 챗봇: [조건 파악] 연령 70세 · 1인 가구 · 월소득 80만원 → [판정] 생계급여 **충족** / 기초연금 **충족** (계산 근거 제시) → [근거] 2026년 기초연금 사업안내 p.23 → [다음 질문] "주거 형태가 자가인가요, 임차인가요?"

1.3 핵심 차별점 — 검색이 아니라 판정

구분	흔한 RAG 챗봇 (검색형)	이 프로젝트 (판정형)
답변 방식	검색 유사도 기반 "추천도 94%"	소득인정액 등 실제 계산 → 충족 / 미충족
정확성 근거	LLM의 그럴듯한 답변	결정론적 계산 코드 + LLM은 설명만 담당
대화 구조	질문 → 검색 → 답변 (단방향)	조건 부족 시 되묻는 루프 (LangGraph)

1.4 왜 이 주제인가

관점	이유
도메인 강점	복지 지식이 있는 개발자는 드묾 — 기술만 있는 지원자와 차별화
기술적 깊이	단순 RAG는 흔한 소재. 판정 로직(계산 엔진) + LangGraph 분기 결합이 차별점
데이터 확보	복지로, 보건복지부 지침서 등 공개 자료가 풍부
사회적 의미	"복잡해서 몰라서 못 받는다"는 실제 문제 해결 시도

1.5 프로젝트 범위 (MVP)

> **범위 제한이 가장 중요합니다.** 처음부터 모든 복지제도를 다루면 절대 못 끝냅니다. 1차 목표는 **기초생활보장 + 긴급복지지원 + 기초연금** 등 2~3개 제도만 판정 로직까지 깊게 구현하는 것입니다. 제도 개수보다 "판정이 실제로 맞는가"가 훨씬 중요합니다.

1.6 사용자 포지셔닝

당사자(복지가 필요한 사람)와 실무자(사회복지사·상담원) 양쪽에 쓸 수 있게 설계합니다. 판정 로직 하나로 "내가 받을 수 있는지"와 "이 사람이 어떤 근거로 대상자가 되는지"를 동시에 answer합니다. 단, **"AI는 참고용 1차 판정이며 최종 확정은 관할 기관이 한다"**는 면책 문구를 반드시 명시합니다.

2 개발 계획서 (단계별 로드맵)

> **마감일 없음.** 아래 "단계"는 강제 일정이 아니라 권장 페이스입니다. 단, **순서는 반드시 지키세요** — 스파이크(0단계)를 건너뛰고 바로 환경 세팅·UI로 가는 것이 가장 흔한 실패 패턴입니다. 각 단계 종료 시 커밋 + README 갱신을 원칙으로 합니다.

단계	목표	주요 작업	산출물
0 최우선	표 파싱 가능성 스파이크	사업안내 PDF 확보 → `table_scout.py`로 표 구조(병합셀·페이지 걸침) 정찰	"판정 데이터화 가능한가" 결론
1	파이썬 + FastAPI 적응	복지제도 목록 CRUD API 구현, Swagger 확인, SQLAlchemy 연동	동작하는 CRUD API
2	데이터 수집·정형화	대상 제도 확정, PDF 텍스트/표 추출, `welfare_services` 테이블 적재	정제 텍스트 + 정형 데이터
3 핵심	판정 엔진 구현	소득인정액 계산 함수, 제도별 선정기준 규칙, 단위 테스트	정확도 검증된 판정 함수
4	기본 RAG 파이프라인	구조 기반 청킹, 임베딩·벡터DB 저장, 근거 인용 답변 생성	질문→근거 답변 (CLI)
5	검색 품질 개선	하이브리드 검색(BM25+벡터), 메타데이터 필터링, Before/After 기록	개선 전후 비교표
6	LangGraph 결합	ChatState 설계, 조건 부족 시 되묻는 루프, 판정 노드 연결	대화형 판정 챗봇
7	도메인 특화 기능	위기 신호 감지, 출처 표시, 환각 방지 프롬프트, 로그 적재	완성된 기능 세트
8	평가 및 UI	RAGAS 평가, 로그 집계 리포트, Streamlit UI	정량 지표가 있는 데모
9	배포 및 문서화	Docker화, 클라우드 배포, README, 데모 영상	완성품 링크
10 선택	Spring Boot 확장	핵심 완성 후 API 게이트웨이/관리자 서버로 Java 결합	Java+Python 통합 사례

2.1 가장 흔한 실패 패턴

- 환경 세팅 → 설계 문서 → 개발 순서로 가다가 3단계쯤에서 "표가 안 뽑히네?"를 뒤늦게 발견
- 제도 개수를 욕심내다 어느 것도 판정 로직까지 못 끝냄

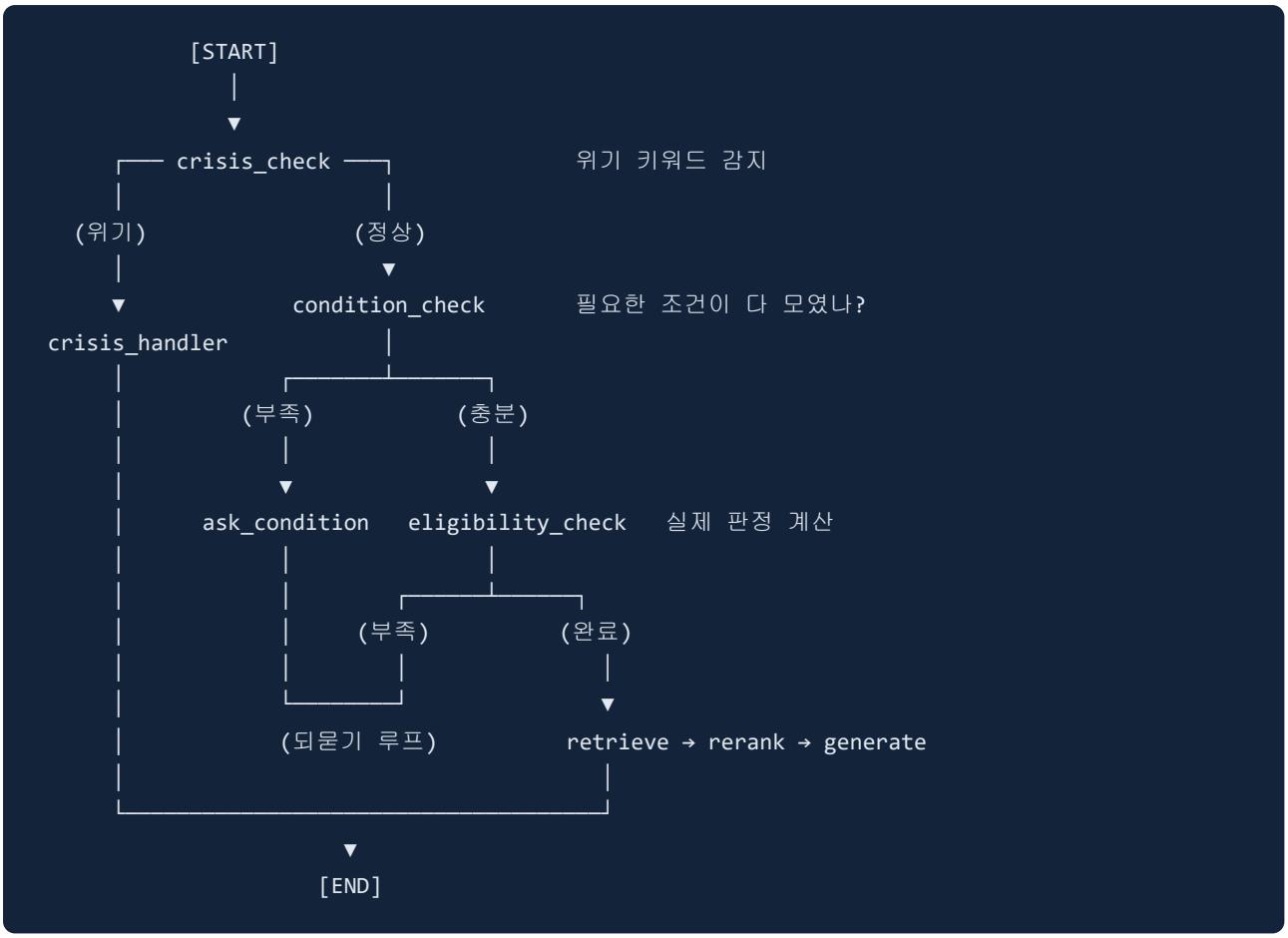
- 마감일이 없다는 이유로 스코프가 끝없이 늘어남 → "판정이 되는 최소 버전 완성"을 최우선 원칙으로 고정

3 시스템 구성

3.1 전체 아키텍처



3.2 LangGraph 대화 흐름



3.3 폴더 구조 (핵심만 발췌)

```
welfare-rag-chatbot/
├── app/
│   ├── main.py                # FastAPI 진입점
│   ├── graph/                 # LangGraph 워크플로 (state, builder, nodes)
│   ├── eligibility/           # 판정 엔진 — income_calculator.py, rules.py
│   ├── rag/                   # 검색 로직 — retriever, reranker, prompts
│   ├── adapters/              # VectorStore / LLM 포트-어댑터
│   └── services/              # chat_service, log_service
├── ingestion/                 # 문서 적재 (독립 실행 스크립트)
├── spike/                     # table_scout.py — 0단계 스파이크
├── data/raw, data/processed
├── evaluation/                # RAGAS 평가
├── frontend/streamlit_app.py
└── tests/                     # test_eligibility.py 등
```

3.4 핵심 데이터 모델

테이블	목적	핵심 필드
`welfare_services`	판정 기준 정형화 (표를 벡터DB 대신 SQL로)	min_age, max_income_ratio, max_asset_amount, household_types, updated_year

`ai_search_logs`	검색/판정 로그 — 성능을 데이터로 설명	question, retrieved_doc_count, response_time_ms, eligibility_result

표(기준 중위소득표 등)는 벡터DB 청크로 넣지 않습니다. 청킹하면 숫자·항목이 깨져 검색이 무의미해지므로, **정형 테이블 + SQL 정확 조회**로 처리합니다.

4 필요한 프로그램 · 도구

4.1 설치가 필요한 프로그램

분류	프로그램	용도	비고
런타임	Python 3.11+	전체 개발 언어	3.13 아님 — 라이브러리 호환성 문제
에디터	VS Code + Python 확장 + Pylance	코드 작성	자바의 IntelliJ 역할
버전관리	Git	형상 관리, 커밋 히스토리가 포트폴리오	GitHub 계정 필요
DB	PostgreSQL (+ pgvector 확장)	정형 데이터 + 벡터 검색 통합	초기엔 SQLite로 대체 가능
컨테이너	Docker Desktop (+ WSL2)	배포용 컨테이너화	9단계에서 필요
빌드 도구	Visual Studio Build Tools (C++ 워크로드)	chromadb 등 설치 시 빌드 에러 방지	윈도우 환경 필수
린터	Ruff	코드 포매팅/린팅	`pip install ruff`

4.2 외부 서비스 / API 키

서비스	필요 시점	비고
OpenAI 또는 Anthropic API 키	4단계(RAG 파이프라인)부터	결제수단 등록 필요, 사용 한도(월 $10 등) 설정 권장
공공데이터포털 — 파일데이터	0단계부터	**키 불필요**, 로그인만 하면 사업안내 PDF 다운로드 가능
공공데이터포털 — OpenAPI	2단계 이후 (선택)	제도 메타데이터 보강용, 활용신청 후 1~2일 소요 — 급하지 않음

4.3 파이썬 패키지 (설치 순서대로)

```
# 0단계 – 스파이크
pip install pdfplumber

# 1단계 – 웹 프레임워크
pip install fastapi uvicorn[standard] python-dotenv pydantic-settings

# 2~3단계 – DB, 문서 처리
pip install sqlalchemy psycopg2-binary pypdf
```

```
# 4단계 — LangChain / 벡터DB / 임베딩
pip install langchain langchain-community langchain-openai langgraph chromadb
pip install sentence-transformers

# 8단계 — UI, 평가, 개발도구
pip install streamlit pytest ruff

pip freeze > requirements.txt
```

> **한 번에 다 설치하지 마세요.** 단계별로 필요할 때 추가하는 것이 의존성 충돌을 줄입니다.

5 필요한 기능 (기능 명세)

5.1 MVP 핵심 기능

#	기능	설명
1	멀티턴 조건 수집	연령·가구원수·소득·재산·거주형태 등을 대화로 파악, 부족하면 되물음
2	제도 검색 및 매칭	수집한 조건에 맞는 제도를 벡터DB에서 검색 (근거 문서용)
3	**자격 판정**	`welfare_services` 기준과 계산 로직으로 충족/미충족/확인필요를 실제 계산 — 이 프로젝트의 핵심
4	근거 인용 답변	어떤 문서 몇 페이지, 어떤 계산식을 썼는지 표시 (환각 방지)
5	위기 신호 감지	자살·학대·긴급 상황 키워드 감지 시 별도 안내 흐름으로 분기
6	신청 절차 안내	필요 서류, 신청 기관, 문의처 제공
7	검색/판정 로그 적재	질문·검색문서수·응답시간·판정결과를 쌓아 성능을 정량적으로 설명

5.2 기능별 구현 우선순위

1. **판정 엔진(기능3)** — 가장 먼저, 가장 정확하게. 이 프로젝트의 차별점이 여기서 결정됨
2. **근거 검색(기능2,4)** — 판정 결과에 신뢰를 더하는 보조 역할
3. **대화 흐름(기능1,5)** — LangGraph로 조건 수집·위기 분기·되묻기 루프 구현
4. **운영 기능(기능6,7)** — 완성도와 정량 근거를 더하는 마무리 단계

5.3 화면 구성 (Streamlit 기준)

- 대화창 — 사용자 입력 + 챗봇 응답 스트리밍
- 판정 결과 카드 — 제도명, 충족 여부, 판정 근거(계산값)
- 출처 패널 — 인용 문서명·페이지 링크
- 면책 문구 — "AI는 참고용 1차 판정이며, 최종 확인은 관할 기관에" 상시 노출

6 처음 시작하는 흐름 (Day-1 체크리스트)

> **원칙**: 환경을 다 세팅하고 시작하지 않습니다. 가장 리스크가 큰 것(표 파싱 가능 여부)부터 작게 찔러보고, 결과에 따라 나머지 계획을 조정합니다.

6.1 0단계 스파이크 실행 순서

- [ ] 공공데이터포털(data.go.kr) 가입 — 파일데이터는 키 불필요
- [ ] Python 3.11 설치
- [ ] 국민기초생활보장 사업안내 PDF 다운로드 → `data/raw/`
- [ ] PDF를 눈으로 훑어 표 구조 확인 (격자 단순한지 / 병합셀 있는지 / 페이지를 넘어가는지)
- [ ] `pip install pdfplumber`
- [ ] `python spike/table_scout.py "국민기초생활보장_사업안내.pdf" --pages 1-50` 실행
- [ ] 결과에 따라 경로 결정 (아래 6.2 표 참고)

6.2 스파이크 결과별 다음 행동

결과	다음 행동
표가 잘 뽑힘 (병합셀 적음)	계획대로 `pdfplumber` 기반 파이프라인 진행
병합셀이 많아 지저분함	`law.go.kr` HTML 경로로 전환 (오히려 더 쉬울 수 있음)
표를 아예 못 찾음	스캔본 PDF 가능성 — OCR 필요 또는 다른 출처 탐색

6.3 1단계 착수 — 헬스체크

```
# app/main.py
from fastapi import FastAPI

app = FastAPI(title="복지제도 판정 챗봇 API")


@app.get("/health")
def health_check():
    return {"status": "ok"}
```

`uvicorn app.main:app --reload` 실행 후 `localhost:8000/docs`가 뜨면 준비 완료입니다.

6.4 첫 커밋

- [ ] `.gitignore` 작성 (`.env` , `venv/` , `.chroma/` , `data/raw/` 포함) — 최우선

- [ ] `.env.example` 작성 (실제 키 없는 템플릿)
- [ ] 폴더 구조 생성
- [ ] 첫 커밋: `chore: 프로젝트 초기 구조 설정`

본 문서는 RAG_사회복지_챗봇개발가이드.md 를 기반으로 기획·계획·구성 흐름만 발췌·재구성한 요약 문서입니다. 상세 구현 코드, 자바↔파이썬 문법 대응, 트러블슈팅, 면접·포트폴리오 어필 포인트는 원본 가이드 문서를 참고하세요.